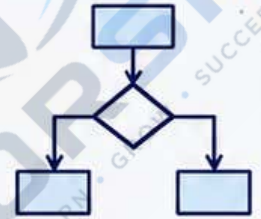


1010101010
0101010101
1010101010
0101010101



INSTRUCTIONS



1. Read the Problem Statement

Understand the objective and expected output before writing the program.



2. Write and Run the Program

Implement the solution, compile and run the program to check the output.



3. Test with Different Inputs

Use multiple test cases to verify the correctness of the program.



4. Add Proper Comments

Include comments to explain the logic and important steps.



5. Maintain the Record

Write the program, output and observations neatly in your practical file.

Code • Practice • Learn • Improve

HAPPY PRACTICALS !



10110
01001
10101



Practical No-02

Aim: Write a Python program to perform basic mathematical operations.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 07/08/2026

Source Code :

```
# Program to perform basic mathematical operations
```

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
print("Addition =", num1 + num2)
```

```
print("Subtraction =", num1 - num2)
```

```
print("Multiplication =", num1 * num2)
```

```
if num2 != 0:
```

```
    print("Division =", num1 / num2)
```

```
else:
```

```
    print("Division is not possible by zero")
```

Output :

```
... Enter first number: 15
Enter second number: 3
Addition = 18.0
Subtraction = 12.0
Multiplication = 45.0
Division = 5.0
```

Practical No-03

Aim: Implement different reshape functions in NumPy such as reshape(), resize() .

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 07/08/2026

Source Code :

```
import numpy as np

# Create a NumPy array
arr = np.array([1, 2, 3, 4, 5, 6])

print("Original Array:")
print(arr)

# Using reshape()
reshaped_arr = arr.reshape(2, 3)

print("\nAfter reshape(2, 3):")
print(reshaped_arr)

# Using resize()
resized_arr = np.resize(arr, (3, 3))

print("\nAfter resize(3, 3):")
print(resized_arr)
```

Output:

```
Original Array:
... [1 2 3 4 5 6]

After reshape(2, 3):
[[1 2 3]
 [4 5 6]]

After resize(3, 3):
[[1 2 3]
 [4 5 6]
 [1 2 3]]
```

Practical No-04

Aim: Implement different ways of concatenating arrays in NumPy using functions.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 10/08/2026

Source Code :

```
import numpy as np
```

```
# Create two arrays
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])
```

```
print("Array 1:")
```

```
print(arr1)
```

```
print("\nArray 2:")
```

```
print(arr2)
```

```
# 1. Using concatenate()
```

```
result1 = np.concatenate((arr1, arr2))
```

```
print("\nUsing concatenate():")
```

```
print(result1)
```

```
# 2. Using hstack()
```

```
result2 = np.hstack((arr1, arr2))
```

```
print("\nUsing hstack():")
```

```
print(result2)
```

```
# 3. Using vstack()
```

```
result3 = np.vstack((arr1, arr2))
```

```
print("\nUsing vstack():")
```

```
print(result3)
```

Output:

```
*** Array 1:
[1 2 3]

Array 2:
[4 5 6]

Using concatenate():
[1 2 3 4 5 6]

Using hstack():
[1 2 3 4 5 6]

Using vstack():
[[1 2 3]
 [4 5 6]]
```

Practical No-05

Aim: Implement array splitting functions in NumPy such as split(), hsplit().

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 10/08/2026

Source Code :

```
import numpy as np

# Create an array
arr = np.array([1, 2, 3, 4, 5, 6])

print("Original Array:")
print(arr)

# 1. Using split()
result1 = np.split(arr, 3)

print("\nUsing split():")
for i in result1:
    print(i)

# 2. Using hsplit()
result2 = np.hsplit(arr, 3)

print("\nUsing hsplit():")
for i in result2:
    print(i)
```

Output:

```
... Original Array:
[1 2 3 4 5 6]

Using split():
[1 2]
[3 4]
[5 6]

Using hsplit():
[1 2]
[3 4]
[5 6]
```

Practical No-06

Aim: Implement different random number generation functions in NumPy using the random module.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 13/08/2026

Source Code :

```
import numpy as np

# Generation of Random Integer

print("Random Integers ")
print(np.random.randint((1,100)))

#Generation of Multiple Random Integers
print("\n Random Integers")
print(np.random.randint((1,100,6)))

#Generation of Random Floating # between 0 & 1

print("\n Floating")
print(np.random.rand())
print("\nRandom Float")
print(np.random.rand(5))

#Specified Range
print("\nRandom Uniform no")
print(np.random.uniform(1,10,7))

#Select a Random elements from an Array

arr = np.array([10,20,30,40,50,60,70])

print("\nRandom Choice ")
```

```
print(np.random.choice(arr))
```

```
np.random.shuffle(arr)
```

```
print("\nSuffled Array")
```

```
print(arr)
```

Output :

```
... Random Integers
[ 0 65]

Random Integers
[ 0 45  1]

Floating
0.6520184617467936

Random Float
[0.67891424 0.65171484 0.61128725 0.9578326 0.81449568]

Random Uniform no
[8.31033251 9.64270275 8.14539436 1.36044747 4.24924732 1.03532379
 8.89672442]

Random Choice
70

Suffled Array
[70 10 40 30 60 50 20]
```

Practical No-07

Aim: To perform various statistical operations on NumPy arrays such as min, max, sum, mean, median, var, std, and percentile.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 13/08/2026

Source Code :

```
# Create an Array
```

```
import numpy as np
```

```
arr = np.array([10,20,30,40,50,60,70,80,90,100])
```

```
print("Array ", arr)
```

```
#Minimum
```

```
print("\nMinimum",np.min(arr))
```

```
#Maximum
```

```
print("\nMaximum",np.max(arr))
```

```
#Mean
```

```
print("\nMean",np.mean(arr))
```

```
#Median
```

```
print("\nMedian",np.median(arr))
```

```
#Standard
```

```
print("\nStandard",np.std(arr))
```

```
#Variance
```

```
print("\nVariance",np.var(arr))
```

```
#Percentile
```

```
print("\nPercentile",np.percentile(arr,50))
```

```
#Sum  
print("\nSum",np.sum(arr))
```

Output:

```
... Array [ 10  20  30  40  50  60  70  80  90 100]  
  
Minimum 10  
  
Maximum 100  
  
Mean 55.0  
  
Median 55.0  
  
Standard 28.722813232690143  
  
Variance 825.0  
  
Percentile 55.0  
  
Sum 550
```

Practical No-08

Aim: To study and implement array creation in NumPy.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 13/08/2026

Source Code :

```
import numpy as np
```

```
# 1D Array
```

```
arr1 = np.array([1,2,3,4,5])
```

```
print("Array from list:")
```

```
print(arr1)
```

```
# 2D Array
```

```
arr2 = np.array([[10,20,30,40],[50,60,70,80]])
```

```
print("\n")
```

```
print(arr2)
```

```
#Array of Zeros
```

```
arr3 = np.zeros(5)
```

```
print("\n")
```

```
print(arr3)
```

```
#Array of Ones
```

```
arr4 = np.ones(5)
```

```
print("\n")
```

```
print(arr4)
```

```
arr5 = np.full(5,7)
```

```
print("\nArray of constant values",arr5)
```

```
arr6 = np.arange(1,11)
```

```
print("\nArray using arrange",arr6)
```

```
arr7 = np.linspace(0,10,5)
print(arr7)

arr8 = np.eye(3)
print("\nIdentity Matrix",arr8)
```

Output:

```
... Array from list:
[1 2 3 4 5]

[[10 20 30 40]
 [50 60 70 80]]

[0. 0. 0. 0. 0.]

[1. 1. 1. 1. 1.]
Array of constant values [7 7 7 7 7]
Array using arrange [ 1  2  3  4  5  6  7  8  9 10]
[ 0.  2.5  5.  7.5 10. ]

Identity Matrix [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
```

Practical No-09

Aim: Access array elements using indexing and slicing techniques.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 19/08/2026

Source Code :

```
import numpy as np

# Create a NumPy array
arr = np.array([10, 20, 30, 40, 50, 60])

print("Original Array:")
print(arr)

# Accessing elements using indexing
print("\nUsing Indexing:")
print("First element:", arr[0])
print("Third element:", arr[2])
print("Last element:", arr[-1])

# Accessing elements using slicing
print("\nUsing Slicing:")
print("First three elements:", arr[0:3])
print("Elements from index 2 to 4:", arr[2:5])
print("Elements from index 1:", arr[1:])
print("Elements up to index 3:", arr[:4])
print("Every second element:", arr[::2])
```

Output:

```
Original Array:
... [10 20 30 40 50 60]

Using Indexing:
First element: 10
Third element: 30
Last element: 60

Using Slicing:
First three elements: [10 20 30]
Elements from index 2 to 4: [30 40 50]
Elements from index 1: [20 30 40 50 60]
Elements up to index 3: [10 20 30 40]
Every second element: [10 30 50]
```

Practical No-10

Aim: To study and implement different arithmetic operations on NumPy arrays, including element-wise operations and scalar operations.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import numpy as np

# Create two NumPy arrays
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([2, 4, 5, 8, 10])

print("Array 1:")
print(arr1)

print("\nArray 2:")
print(arr2)

# Element-wise Addition
print("\nElement-wise Addition:")
print(arr1 + arr2)

# Element-wise Subtraction
print("\nElement-wise Subtraction:")
print(arr1 - arr2)

# Element-wise Multiplication
print("\nElement-wise Multiplication:")
print(arr1 * arr2)

# Element-wise Division
print("\nElement-wise Division:")
print(arr1 / arr2)

# Element-wise Modulus
print("\nElement-wise Modulus:")
print(arr1 % arr2)
```

```
# Element-wise Power
print("\nElement-wise Power:")
print(arr1 ** 2)

# Scalar Operations
scalar = 5

print("\nScalar Value:", scalar)

print("\nAddition with Scalar:")
print(arr1 + scalar)

print("\nSubtraction with Scalar:")
print(arr1 - scalar)

print("\nMultiplication with Scalar:")
print(arr1 * scalar)

print("\nDivision with Scalar:")
print(arr1 / scalar)
```

Output:

```
... Array 1:
[10 20 30 40 50]

Array 2:
[ 2  4  5  8 10]

Element-wise Addition:
[12 24 35 48 60]

Element-wise Subtraction:
[ 8 16 25 32 40]

Element-wise Multiplication:
[ 20  80 150 320 500]

Element-wise Division:
[5.  5.  6.  5.  5.]

Element-wise Modulus:
[0 0 0 0 0]

Element-wise Power:
[ 100  400  900 1600 2500]

Scalar Value: 5

Addition with Scalar:
[15 25 35 45 55]

Subtraction with Scalar:
[ 5 15 25 35 45]

Multiplication with Scalar:
[ 50 100 150 200 250]

Division with Scalar:
[ 2.  4.  6.  8. 10.]
```

Practical No-11

Aim: To study and implement different matrix operations in NumPy.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import numpy as np
```

```
# Create two matrices
```

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
B = np.array([[5, 6],  
              [7, 8]])
```

```
print("Matrix A:")  
print(A)
```

```
print("\nMatrix B:")  
print(B)
```

```
# Matrix Addition  
print("\nMatrix Addition:")  
print(A + B)
```

```
# Matrix Subtraction  
print("\nMatrix Subtraction:")  
print(A - B)
```

```
# Matrix Multiplication  
print("\nMatrix Multiplication:")  
print(np.dot(A, B))
```

```
# Element-wise Multiplication  
print("\nElement-wise Multiplication:")  
print(A * B)
```

```
# Transpose of Matrix A  
print("\nTranspose of Matrix A:")
```

```
print(A.T)
```

```
# Determinant of Matrix A
```

```
print("\nDeterminant of Matrix A:")
```

```
print(np.linalg.det(A))
```

```
# Inverse of Matrix A
```

```
print("\nInverse of Matrix A:")
```

```
print(np.linalg.inv(A))
```

Output :

```
*** Matrix A:
[[1 2]
 [3 4]]

Matrix B:
[[5 6]
 [7 8]]

Matrix Addition:
[[ 6  8]
 [10 12]]

Matrix Subtraction:
[[-4 -4]
 [-4 -4]]

Matrix Multiplication:
[[19 22]
 [43 50]]

Element-wise Multiplication:
[[ 5 12]
 [21 32]]

Transpose of Matrix A:
[[1 3]
 [2 4]]

Determinant of Matrix A:
-2.0000000000000004

Inverse of Matrix A:
[[-2.  1.]
 [ 1.5 -0.5]]
```

Practical No-12

Aim: To study and implement different string operations in NumPy using the numpy.char module.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import numpy as np

# Create string arrays
str1 = np.array(["Hello", "Python", "NumPy"])
str2 = np.array(["World", "Programming", "Library"])

print("String Array 1:")
print(str1)

print("\nString Array 2:")
print(str2)

# 1. String Concatenation
print("\nString Concatenation:")
print(np.char.add(str1, str2))

# 2. Convert to Uppercase
print("\nUppercase:")
print(np.char.upper(str1))

# 3. Convert to Lowercase
print("\nLowercase:")
print(np.char.lower(str1))

# 4. String Length
print("\nString Length:")
print(np.char.str_len(str1))

# 5. String Replacement
print("\nString Replacement:")
print(np.char.replace(str1, "o", "O"))
```

```
# 6. String Capitalization
print("\nCapitalized:")
print(np.char.capitalize(str1))
```

```
# 7. String Title
print("\nTitle Case:")
print(np.char.title(str2))
```

```
# 8. String Comparison
print("\nString Comparison:")
print(np.char.equal(str1, str2))
```

Output:

```
*** String Array 1:
['Hello' 'Python' 'NumPy']

String Array 2:
['World' 'Programming' 'Library']

String Concatenation:
['HelloWorld' 'PythonProgramming' 'NumPyLibrary']

Uppercase:
['HELLO' 'PYTHON' 'NUMPY']

Lowercase:
['hello' 'python' 'numpy']

String Length:
[5 6 5]

String Replacement:
['Hello' 'PythOn' 'NumPy']

Capitalized:
['Hello' 'Python' 'Numpy']

Title Case:
['World' 'Programming' 'Library']

String Comparison:
[False False False]
```

Practical No-13

Aim: To study and implement sorting and searching operations in NumPy arrays using functions.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import numpy as np

# Create a NumPy array
arr = np.array([50, 20, 80, 10, 40, 30])

print("Original Array:")
print(arr)

# 1. Sorting using np.sort()
sorted_arr = np.sort(arr)

print("\nSorted Array:")
print(sorted_arr)

# 2. Finding the index of the minimum value
min_index = np.argmin(arr)

print("\nIndex of Minimum Element:")
print(min_index)

# 3. Finding the index of the maximum value
max_index = np.argmax(arr)

print("\nIndex of Maximum Element:")
print(max_index)

# 4. Searching for an element using np.where()
search_value = 40
position = np.where(arr == search_value)

print("\nPosition of", search_value, ":")
```

```
print(position)
```

```
# 5. Searching using np.searchsorted()
```

```
sorted_array = np.sort(arr)
```

```
position2 = np.searchsorted(sorted_array, 40)
```

```
print("\nPosition returned by searchsorted() for 40:")
```

```
print(position2)
```

Output:

```
*** Original Array:
[50 20 80 10 40 30]

Sorted Array:
[10 20 30 40 50 80]

Index of Minimum Element:
3

Index of Maximum Element:
2

Position of 40 :
(array([4]),)

Position returned by searchsorted() for 40:
3
```

Practical No-14

Aim: To study and implement creation of Series in Pandas and access their elements using indexing and slicing.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import pandas as pd

# Create a Pandas Series
data = pd.Series([10, 20, 30, 40, 50])

print("Original Series:")
print(data)

# Accessing elements using position indexing
print("\nFirst Element:")
print(data.iloc[0])

print("\nThird Element:")
print(data.iloc[2])

# Accessing elements using label indexing
print("\nElement with Index 3:")
print(data.loc[3])

# Slicing using position
print("\nFirst Three Elements:")
print(data.iloc[0:3])

# Slicing using label
print("\nElements from Index 1 to 3:")
print(data.loc[1:3])
```

Output:

```
*** Original Series:
0    10
1    20
2    30
3    40
4    50
dtype: int64

First Element:
10

Third Element:
30

Element with Index 3:
40

First Three Elements:
0    10
1    20
2    30
dtype: int64

Elements from Index 1 to 3:
1    20
2    30
3    40
dtype: int64
```

Practical No-15

Aim: To study and perform basic operations on Pandas DataFrame. .

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date:20/08/2026

Source Code :

```
import pandas as pd

# Create a DataFrame
data = {
    "Name": ["Shahid", "Faizan", "Rahul", "Aman"],
    "Age": [20, 21, 19, 22],
    "Marks": [85, 90, 78, 88]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Display first two rows
print("\nFirst Two Rows:")
print(df.head(2))

# Display last two rows
print("\nLast Two Rows:")
print(df.tail(2))

# Access a column
print("\nNames:")
print(df["Name"])

# Access multiple columns
print("\nName and Marks:")
print(df[["Name", "Marks"]])

# Add a new column
df["Result"] = ["Pass", "Pass", "Pass", "Pass"]
```

```
print("\nAfter Adding Result Column:")
print(df)
```

```
# Find average marks
print("\nAverage Marks:")
print(df["Marks"].mean())
```

```
# Find maximum marks
print("\nMaximum Marks:")
print(df["Marks"].max())
```

```
# Find minimum marks
print("\nMinimum Marks:")
print(df["Marks"].min())
```

```
# Display DataFrame information
print("\nDataFrame Shape:")
print(df.shape)
```

Output:

```
... Original DataFrame:
   Name  Age  Marks
0  Shahid   20    85
1  Faizan   21    90
2   Rahul   19    78
3   Aman   22    88

First Two Rows:
   Name  Age  Marks
0  Shahid   20    85
1  Faizan   21    90

Last Two Rows:
   Name  Age  Marks
2  Rahul   19    78
3   Aman   22    88

Names:
0  Shahid
1  Faizan
2   Rahul
3   Aman
Name: Name, dtype: object

Name and Marks:
   Name  Marks
0  Shahid    85
1  Faizan    90
2   Rahul    78
3   Aman    88

After Adding Result Column:
   Name  Age  Marks  Result
0  Shahid   20    85    Pass
1  Faizan   21    90    Pass
2   Rahul   19    78    Pass
3   Aman   22    88    Pass

Average Marks:
85.25

Maximum Marks:
90

Minimum Marks:
78

DataFrame Shape:
(4, 4)
```

Practical No-16

Aim: To study and implement data selection, filtering, and conditional operations on Pandas DataFrames.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import pandas as pd

data = {
    "Name": ["Amit", "Rahul", "Priya", "Neha", "Karan"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [85, 65, 92, 78, 55]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Data Selection
print("\nSelected Name Column:")
print(df["Name"])

print("\nSelected Name and Marks:")
print(df[["Name", "Marks"]])

# Data Filtering
print("\nStudents with Marks greater than 75:")
print(df[df["Marks"] > 75])

# Conditional Operations
print("\nStudents with Marks >= 80:")
print(df[df["Marks"] >= 80])

print("\nStudents with Age = 20:")
print(df[df["Age"] == 20])

# Multiple Conditions
```

```
print("\nStudents with Marks > 70 and Age < 22:")
print(df[(df["Marks"] > 70) & (df["Age"] < 22)])
```

Output :

```
... Original DataFrame:
   Name  Age  Marks
0  Amit   20    85
1  Rahul  21    65
2  Priya  19    92
3  Neha   22    78
4  Karan  20    55

Selected Name Column:
0  Amit
1  Rahul
2  Priya
3  Neha
4  Karan
Name: Name, dtype: object

Selected Name and Marks:
   Name  Marks
0  Amit    85
1  Rahul   65
2  Priya   92
3  Neha    78
4  Karan    55

Students with Marks greater than 75:
   Name  Age  Marks
0  Amit   20    85
2  Priya  19    92
3  Neha   22    78

Students with Marks >= 80:
   Name  Age  Marks
0  Amit   20    85
2  Priya  19    92

Students with Age = 20:
   Name  Age  Marks
0  Amit   20    85
4  Karan  20    55

Students with Marks > 70 and Age < 22:
   Name  Age  Marks
0  Amit   20    85
2  Priya  19    92
```

Practical No-17

Aim: To study and implement techniques for detecting, removing, and replacing missing data (NaN values) in Pandas DataFrames

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Amit", "Rahul", "Priya", "Neha"],
    "Age": [20, np.nan, 19, 22],
    "Marks": [85, 78, np.nan, 90]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Detect missing values
print("\nMissing Values:")
print(df.isnull())

print("\nTotal Missing Values:")
print(df.isnull().sum())

# Remove rows containing missing values
print("\nAfter Removing Missing Rows:")
print(df.dropna())

# Replace missing values with 0
print("\nAfter Replacing Missing Values with 0:")
print(df.fillna(0))

# Replace missing values with column mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
print("\nAfter Replacing Missing Values with Mean:")
print(df)
```

Output:

```
*** Original DataFrame:
   Name  Age  Marks
0  Amit  20.0  85.0
1  Rahul NaN  78.0
2  Priya 19.0   NaN
3  Neha  22.0  90.0

Missing Values:
   Name  Age  Marks
0  False False False
1  False  True False
2  False False  True
3  False False False

Total Missing Values:
Name      0
Age       1
Marks     1
dtype: int64

After Removing Missing Rows:
   Name  Age  Marks
0  Amit  20.0  85.0
3  Neha  22.0  90.0

After Replacing Missing Values with 0:
   Name  Age  Marks
0  Amit  20.0  85.0
1  Rahul  0.0  78.0
2  Priya 19.0  0.0
3  Neha  22.0  90.0

After Replacing Missing Values with Mean:
   Name      Age      Marks
0  Amit  20.000000  85.000000
1  Rahul  20.333333  78.000000
2  Priya  19.000000  84.333333
3  Neha  22.000000  90.000000
```

Practical No-18

Aim: To study and implement reading and writing operations in Pandas using different file formats such as CSV, Excel, and JSON.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date:20/08/2026

Source Code :

```
import pandas as pd

# Create a DataFrame
data = {
    "Name": ["Amit", "Rahul", "Priya"],
    "Age": [20, 21, 19],
    "Marks": [85, 78, 92]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Write CSV file
df.to_csv("students.csv", index=False)

# Read CSV file
csv_data = pd.read_csv("students.csv")

print("\nData read from CSV:")
print(csv_data)

# Write Excel file
df.to_excel("students.xlsx", index=False)

# Read Excel file
excel_data = pd.read_excel("students.xlsx")

print("\nData read from Excel:")
print(excel_data)
```

```
# Write JSON file
df.to_json("students.json", orient="records")

# Read JSON file
json_data = pd.read_json("students.json")

print("\nData read from JSON:")
print(json_data)
```

Output:

```
*** Original DataFrame:
   Name  Age  Marks
0  Amit   20    85
1  Rahul  21    78
2  Priya  19    92

Data read from CSV:
   Name  Age  Marks
0  Amit   20    85
1  Rahul  21    78
2  Priya  19    92

Data read from Excel:
   Name  Age  Marks
0  Amit   20    85
1  Rahul  21    78
2  Priya  19    92

Data read from JSON:
   Name  Age  Marks
0  Amit   20    85
1  Rahul  21    78
2  Priya  19    92
```

Practical No-19

Aim: To study and implement data visualization using Pandas plotting functions.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date:20/08/2026

Source Code :

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    "Month": ["January", "February", "March", "April", "May"],
    "Sales": [100, 150, 120, 180, 200]
}

df = pd.DataFrame(data)

print("DataFrame:")
print(df)

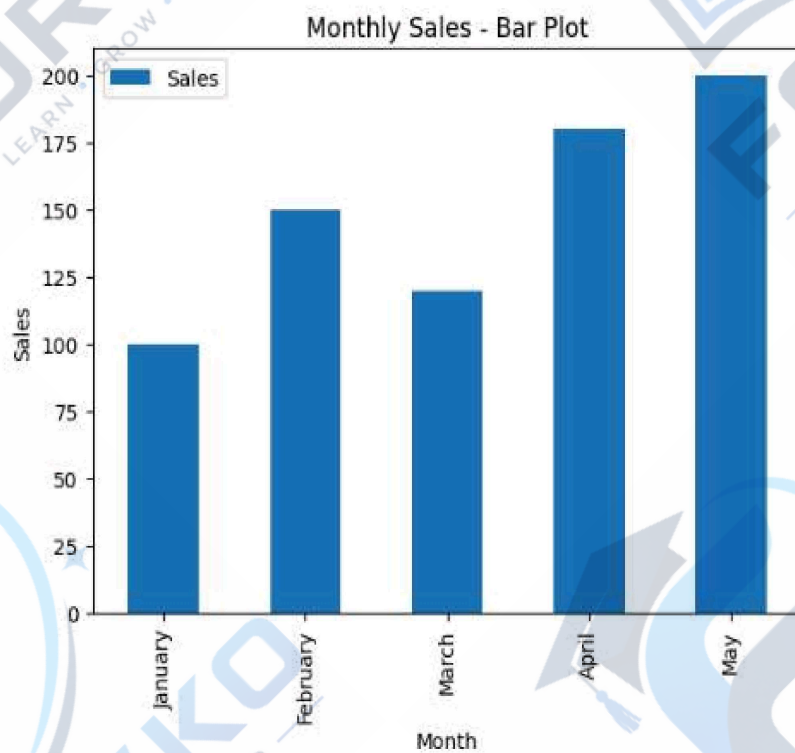
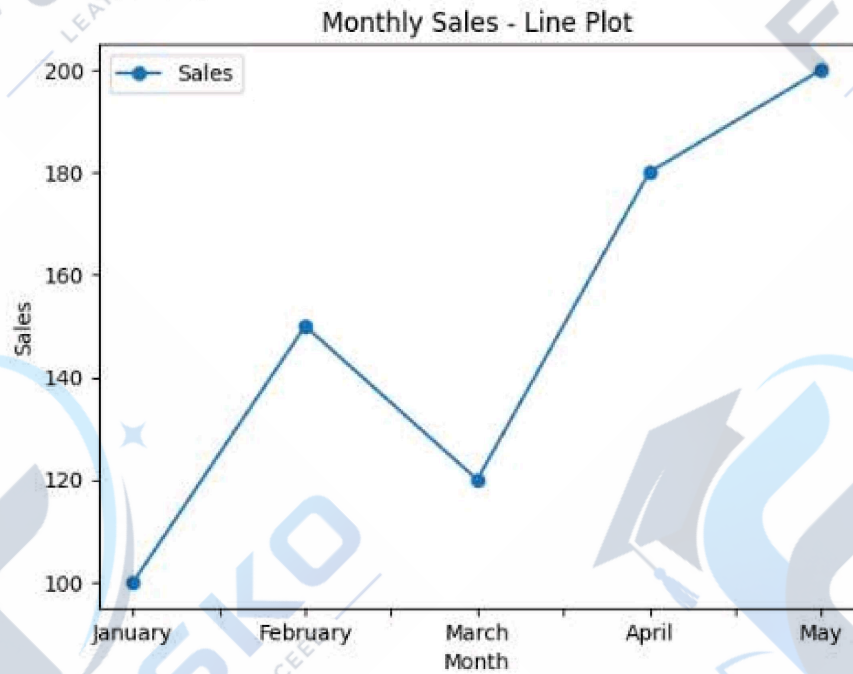
# Line Plot
df.plot(x="Month", y="Sales", kind="line", marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

# Bar Plot
df.plot(x="Month", y="Sales", kind="bar")
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

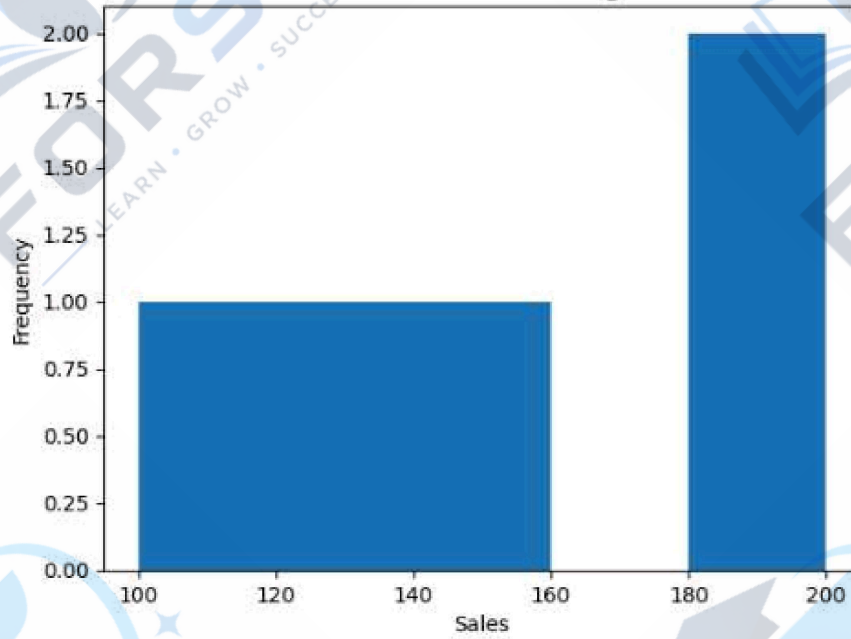
# Histogram
df["Sales"].plot(kind="hist", bins=5)
plt.title("Sales Distribution - Histogram")
plt.xlabel("Sales")
plt.show()
```

Output:

```
*** DataFrame:
   Month  Sales
0  January   100
1  February  150
2   March   120
3   April   180
4    May    200
```



Sales Distribution - Histogram



Practical No-20

Aim: To study and implement data visualization using Matplotlib.

Name: Abdul Shahid

Class: B.Sc -III (SEM-V)

Subject: Computer Science

Id No: 71607

Date: 20/08/2026

Source Code :

```
import matplotlib.pyplot as plt
```

```
# Data
```

```
months = ["January", "February", "March", "April", "May"]
```

```
sales = [100, 150, 120, 180, 200]
```

```
# 1. Line Plot
```

```
plt.plot(months, sales, marker="o")
```

```
plt.title("Monthly Sales - Line Plot")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
# 2. Bar Chart
```

```
plt.bar(months, sales)
```

```
plt.title("Monthly Sales - Bar Chart")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
# 3. Scatter Plot
```

```
plt.scatter(months, sales)
```

```
plt.title("Monthly Sales - Scatter Plot")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
# 4. Pie Chart
```

```
plt.pie(sales, labels=months, autopct="%1.1f%%")
```

```
plt.title("Monthly Sales - Pie Chart")
```

```
plt.show()
```

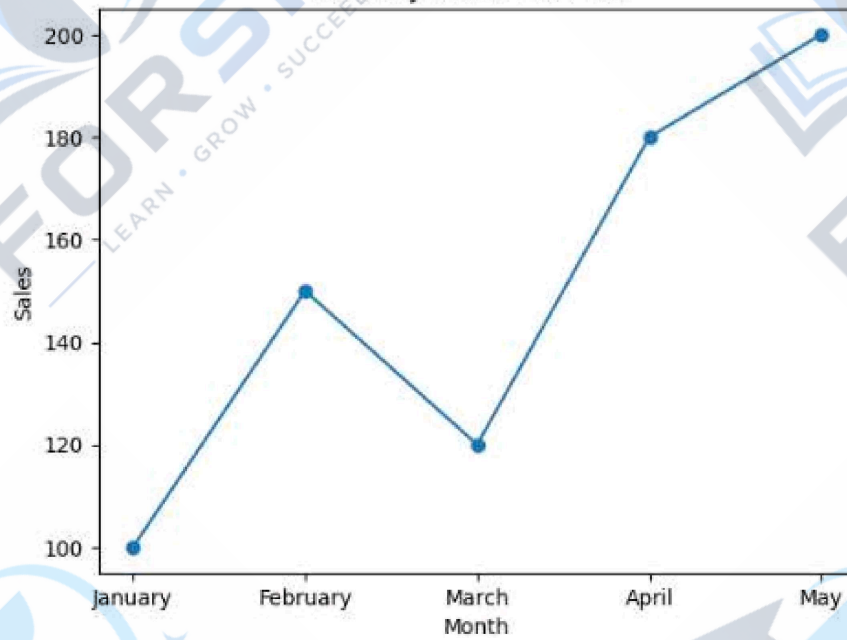
```
# 5. Histogram
```

```
plt.hist(sales, bins=5)
plt.title("Sales Distribution - Histogram")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

Output:

...

Monthly Sales - Line Plot



Monthly Sales - Bar Chart

